

Mandarin-English Neural Machine Translation

Haley F. Chen

Jacobs School of Engineering, UC San Diego

haleyfchen@gmail.com

GitHub: haelyra/Mandarin-to-English-Transformer

Abstract

Building on the transformative insight that “attention is all you need,” this project implements a complete Transformer-based neural machine translation system from scratch in PyTorch, reproducing the encoder-decoder architecture introduced by Vaswani et al. [1] for the linguistically challenging Mandarin-to-English direction. Trained on a carefully cleaned and aligned subset of 43,000 sentence pairs from the ShareGPT Chinese-English parallel corpus, the model, featuring 3-layer encoder and decoder stacks with 8 attention heads and 256-dimensional embeddings, achieves a training perplexity of 15.41 and validation perplexity of 20.36 after 10 epochs. The end-to-end pipeline includes custom tokenization (jieba [2] for Chinese, word-level for English), vocabulary construction, checkpointing, and greedy decoding, all of which are designed for full reproducibility with configurable artifacts that enable deterministic reruns and straightforward extensions in future experiments.

1. Problem Statement and Goals

Neural machine translation converts a source-language sequence into a target-language sequence while preserving meaning and grammaticality. This project aims to: (1) implement an encoder-decoder Transformer from first principles (multi-head attention, positional encoding, feed-forward blocks), (2) build an end-to-end data and training pipeline, and (3) evaluate learning progress with a reproducible experimental setup.

2. Dataset and Preprocessing

Input format and filtering. Parallel data are read from TSV files via a header-aware loader (CSV DictReader with tab delimiter), selecting source/target columns (default: zh/en). Empty and mismatched-length source/target pairs are skipped, along with duplicates and sentences exceeding 100 tokens to prevent noisy or degenerate training examples.

Tokenization. English text is lowercased, normalized (punctuation attached to words), and tokenized via whitespace splitting with subword BPE (SentencePiece, 32k ops). Chinese text uses jieba word segmentation followed by BPE when available; otherwise falls back to character-level tokenization with explicit spacing.

Vocabulary and special symbols. Vocabularies are built separately for source/target from training tokens

using frequency filtering (`min_freq=5`) and BPE merging. Special tokens include `<BOS>`, `<EOS>`, `<PAD>`, and `<UNK>`. Sequences are numericalized with explicit BOS/EOS, truncated/padded to `max_len` (default 128), discarding examples where `len(BOS + tokens + EOS)` are greater than `max_len`.

Batching and padding. Batches are grouped by length buckets then dynamically right-padded to the longest sequence in the batch using `<PAD>` (index 0). Padded tensors are returned as integer arrays of shape (N, T) for source/target, with a separate mask tensor indicating valid positions.

Table 1: Data and vocabulary summary

Train pairs (subset)	43,000
Validation pairs (subset)	4,800
Max sequence length	64
Min token frequency	2
Zh vocab size	19,098
En vocab size	25,515

3. Model Architecture

Core module. The system wraps PyTorch’s `nn.Transformer` with `batch_first=False`, so internal attention modules operate over tensors shaped (T, N, d) . Inputs are embedded as (N, T, d) and transposed to match the expected ordering before encoder/decoder passes.

Embeddings. Source and target tokens use separate learned embedding tables. Positional information is represented by a learned positional embedding table (`nn.Embedding(max_len, d)`), added to token embeddings for both source and target.

Weight tying. The output projection (`generator`) is tied to the target embedding matrix (specifically, logits are produced with shared parameters), reducing parameter count and encouraging consistency between embedding and softmax spaces.

Initialization. Embedding tables are initialized with Xavier/Glorot uniform initialization.

Table 2: Model and training configuration

Encoder / decoder layers	3 / 3
Attention heads	8
Embedding dim	256
FFN dim	512
Dropout	0.1
Batch size	32
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$)
Learning rate	3×10^{-4}
LR schedule	ReduceLROnPlateau (factor 0.7, patience 1)
Parameters	15,394,560

4. Training Setup

Teacher forcing objective. Training uses a standard next-token objective: the decoder is fed `tgt[:, :-1]` and predicts `tgt[:, 1:]`. Loss is computed with `CrossEntropyLoss` ignoring padding tokens and using label smoothing (`label_smoothing=0.1`).

Masks. The decoder uses a causal (upper-triangular) subsequent mask to prevent attending to future tokens, and key padding masks are constructed from `locations` for both source and target.

Optimization and stability. The optimizer used is the Adam optimizer with $(\beta_1, \beta_2) = (0.9, 0.98)$ and $\epsilon = 10^{-9}$. Gradients are clipped to a max norm of 1.0 prior to the optimizer step.

Scheduling and early stopping. The learning rate is adapted with `ReduceLROnPlateau` (factor 0.7, patience 1) using validation loss as the monitored metric, and training halts early after a specified number of epochs that do not improve.

Checkpointing. On validation improvement, a checkpoint is saved containing model weights, both vocabularies, and the full run configuration (CLI args), supporting exact reruns.

5. Results

The primary quantitative metric is perplexity (computed as $\exp(\text{loss})$). The model shows substantial training progress across epochs, reaching a 15.41 train perplexity and 20.36 validation perplexity at epoch 10.

Table 3: Final perplexity after 10 epochs.

Split	Loss	Perplexity
Train	2.7353	15.41
Validation	3.0134	20.36

Greedy decoding. At inference time, decoding begins with `and` and repeatedly appends the argmax token from the output distribution until reaching `(or max_len)`. Source padding masks are reused to constrain encoder–decoder attention.

6. Limitations and Next Steps

Evaluation. Perplexity measures next-token predictive fit but does not fully capture translation quality. A next step is to add BLEU evaluation with a standardized, reproducible implementation (for example, SacreBLEU) and clearly report tokenization settings Post [3].

Modeling. Improve decoding (beam search), experiment with max sequence length, and refine tokenization/vocabulary (for example, subword methods) to reduce `<unk>` and improve fluency.

Data. Expand training beyond the 43K subset and add stricter cleaning and domain filtering to reduce noise.

7. Reproducibility

The repository includes scripts and notebooks for dataset preparation (JSON to TSV), vocabulary construction, training with validation, checkpoint saving, and greedy decoding. Hyperparameters and configuration choices are recorded to support deterministic reruns and follow-on experiments, consistent with reproducibility best practices encouraged by major ML venues. [1]

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.
- [2] Junyi Sun. Jieba chinese text segmentation. *GitHub repository*, 2014. Accessed: 2025.
- [3] Matt Post. A call for clarity in reporting bleu scores. In *Proceedings of the Third Conference on Machine Translation (WMT)*, 2018.